Deep Learning

Technical Report

By: Hala Nazzal

Course: Deep Learning – Spring 2025/2026

Course instructor: Dr. Ala’a Al-Habashna

Developing a Deep-Learning system for a Computer Vision Application

1 Problem statement

Cataracts, diabetic retinopathy and glaucoma these the three causes that we did find that causes preventable blindness in the world. The manual diagnosis by ophthalmologists is time consuming and error prone . The purpose of this project is to develop an image classifcation system based on deep learning for automatic classification of retinal fundus images into any of the four classes i.e. Cataract, Diabetic Retinopathy, Glaucoma or Normal.. (Ivana Lucia Kharisma, 2025)

(1) Input: image of retinal fundus Output: One of four types of disease. (2) Investigate and evaluate different CNN architectures. (3) Use appropriate evaluation metrics to achieve good classification accuracy. (4) Deploy the best model to a prediction UI.

2 Research on the Neural Networks and architectures

2.1 Neural Networks used for the problem

Convolutional Neural Networks (CNNs) is the mainstream deep learning method for image classification task. Because they automatically learn spatial hierarchies of features using convolutional layers, pooling layers and fully connected layers, CNNs are well suited to retinal image analysis.

CNNs are preferred for classification of eye diseases as fine-grained features in retinal images such as blood vessel patterns, optic disc morphology and lesion textures need hierarchical feature extraction at multiple scales. We selected four architectures: Custom CNN (3-block from scratch), Deeper CNN (4-block VGG inspired), AlexNet (pretrained on ImageNet) and ResNet18 (from scratch with residual connections).

2.2 Modern architectures

AlexNet was the first deep CNN to win the ImageNet competition (Team, 2020). It was composed of 5 convolutional layers and 3 fully connected layers with ReLU activation and dropout regularization. VGGNet (Simonyan and Zisserman, 2015) goes even further with a deeper stack of very small 3x3 filters, demonstrating that depth is indeed a key factor.

Variants of ResNet and VGG-style networks are the most widely used architectures in the literature for medical image classification tasks (e.g., eye disease detection), mainly because of their powerful feature extraction capabilities and availability of pre-trained weights.

2.3 Modern architectures comparison

Table 1: Modern architectures used to solve the problem.

Architecture	Description and types of layers	Advantages	Disadvantages
Custom CNN	Custom CNN 3 conv blocks (Conv2d-BN-ReLU-MaxPool), 2 FC layers 224x224x3 input, 32/64/128 filters, FC 256->4 Simple.	Simple, fast to train, low memory	Easy to train Limited memory capacity May miss complex patterns .
Deeper CNN	Deeper CNN 4 conv blocks with 2 conv layers each (VGG style), 2 FC layers. Filters: 32/64/128/256, FC: 512 -> 4.	Better feature extraction, strong on structured patterns	Slower training, more prone to overfitting
AlexNet Pretrained	5 conv layers + 3 FC layers. ImageNet pretrained weights; final FC replaced (4096->4). Feature layers frozen.	Fast convergence, leverages ImageNet features	Less accuracy when only last layer is trained Feature layers suited for non-retinal domain
ResNet18 Scratch	18-layer residual network with skip connections. Trained from random initialisation. Final FC: 512->4.	Highest accuracy, least parameters (11.2M), no vanishing gradients.	Some overfitting. Requires more data for optimal performance.

3 Models' development and training

3.1 Dataset

The data set "Eye Diseases Classification" was downloaded from Kaggle ((·, 2022)). It contains 4,217 color retinal fundus images with four classes i.e. Cataract (1,038), Diabetic Retinopathy (1,098), Glaucoma (1,007) and Normal (1,074 Images were taken with a standard fundus lens and resized to 224×224 pixels. All images are normalized per channel to mean 0.5 and std 0.5.

The data set was split into three subsets: Training set: 2,951 images (70%) for revision of model parameters. Validation set: 632 images (15%) for hyper-parameter tuning Test set: 634 images (15%) completely withheld for final unbiased evaluation The split was random, with PyTorch random_split with fixed seed (42) for reproducibility.

3.2 Training and validation

We tested three combinations of hyperparameters for each architecture (12 experiments in total). The loss function was cross-entropy loss. Table 2 and 3 represent the accuracies of all examined combinations of hyperparameters on training set and validation set respectively.

From the result that I did work on for each models Best combos were

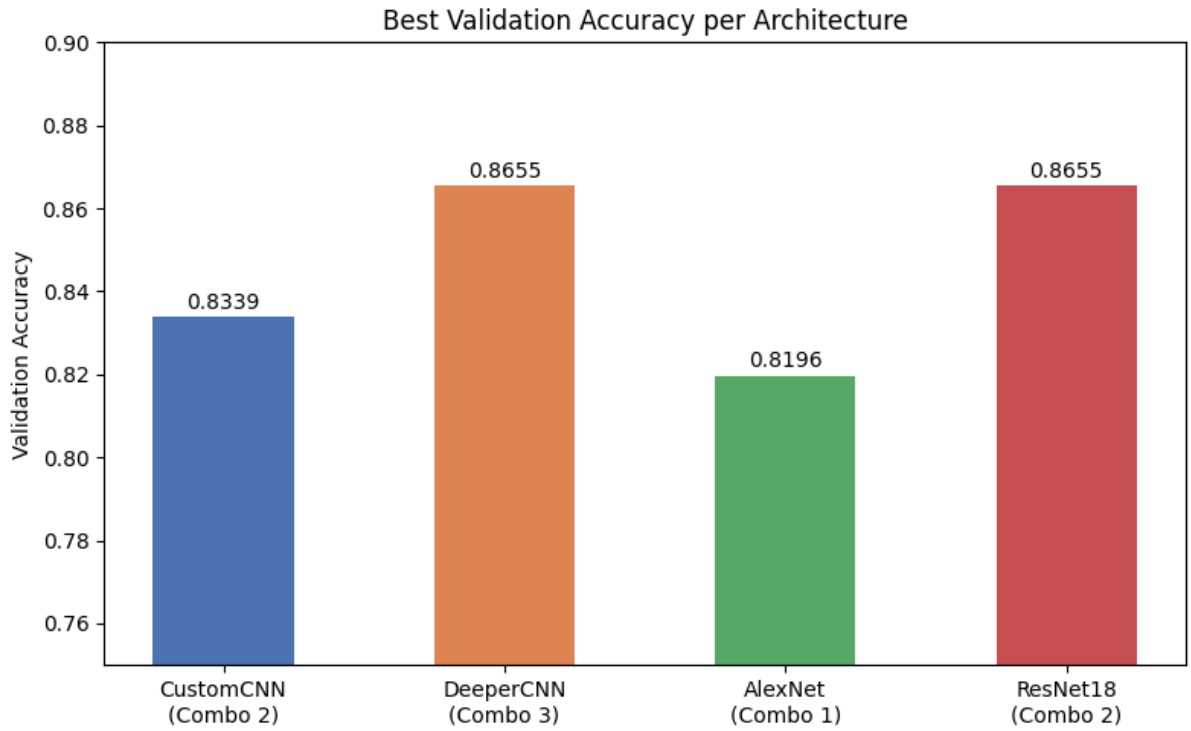


Table 2: Summary of hyperparameters considered per architecture

Architecture	Hyper-parameter	Description	Value(s)
Architecture 1 Custom CNN	**Learning Rate**	Controls step size during gradient descent	0.001 / 0.0001
	Optimizer	Algorthm used to update model weights	Adam / SGD
Architecture 2 Deeper CNN	**Learning Rate**	Controls step size during gradient descent	0.001 / 0.0001
	Dropout	Fraction of neurons dropped to prevent overfitting	0.3 / 0.5
Architecture 3 AlexNet	**Learning Rate**	Controls step size during gradient descent	0.001 / 0.0001
	Optimizer	Algorithm used to update model weight	Adam / SGD
Architecture 4 ResNet18	**Learning Rate**	learning rate step size	0.001 / 0.0001
	Optimizer	Algorithm used to update the weightss of the model	Adam / SGD

Table 3: Combinations of the hyperparameter values and corresponding performance achieved.

Architecture	Combination of hyperparameter values	Training performance	Validation performance
Architecture 1 Custom CNN	Combo 1: Adam, lr=0.001, dropout=0.5	Train Acc: 0.6391	Val Acc: 0.6313
	Combo 2 (Best): Adam, lr=0.0001, dropout=0.5	Train Acc: 0.8834	Val Acc: 0.8339
	Combo 3: SGD, lr=0.001, dropout=0.3	Train Acc: 0.8526	Val Acc: 0.7880
Architecture 2 Deeper CNN	Combo 1: Adam, lr=0.001, dropout=0.5	Train Acc: 0.6723	Val Acc: 0.6139
	Combo 2: Adam, lr=0.0001, dropout=0.5	Train Acc: 0.9234	Val Acc: 0.8497
	Combo 3 (Best): SGD, lr=0.001, dropout=0.3	Train Acc: 0.8983	Val Acc: 0.8655
Architecture 3 AlexNet	Combo 1 (Best): Adam, lr=0.001	Train Acc: 0.7699	Val Acc: 0.8196
	Combo 2: Adam, lr=0.0001	Train Acc: 0.7791	Val Acc: 0.7816
	Combo 3: SGD, lr=0.001	Train Acc: 0.7076	Val Acc: 0.6867
Architecture 4 ResNet18	Combo 1: Adam, lr=0.001	Train Acc: 0.8048	Val Acc: 0.7801
	Combo 2 (Best): Adam, lr=0.0001	Train Acc: 0.9732	Val Acc: 0.8655
	Combo 3: SGD, lr=0.001	Train Acc: 0.8397	Val Acc: 0.8149

4 Models' testing and evaluation

4.1 Testing

The best model for each architecture was tested on the hold-out test set (634 images). The model was evaluated on the basis of four macro averaged metrics i.e. Accuracy, Precision, Recall and F1 Score. We decided to use macro-averaging in order to give equal weight to the four disease classes . Details are given in Table 4.

Table 4: Values of the evaluation metrics obtained in the test set for the best model of each architecture

Architecture	Best combination of hyper-parameter values	Values obtained for the evaluation metrics
Custom CNN	Combo 2: Adam, lr=0.0001, dropout=0.5	Acc=0.8328, Prec=0.8349,

		Recall=0.8245, F1=0.8223
Deeper CNN	Combo 3: SGD, lr=0.001, dropout=0.3	Acc=0.8407, Prec=0.8551, Recall=0.8444, F1=0.8404
AlexNet Pretrained	Combo 1: Adam, lr=0.001	Acc=0.7744, Prec=0.8271, Recall=0.7570, F1=0.7486
ResNet18 Scratch	Combo 2: Adam, lr=0.0001	Acc=0.8502, Prec=0.8589, Recall=0.8439, F1=0.8444

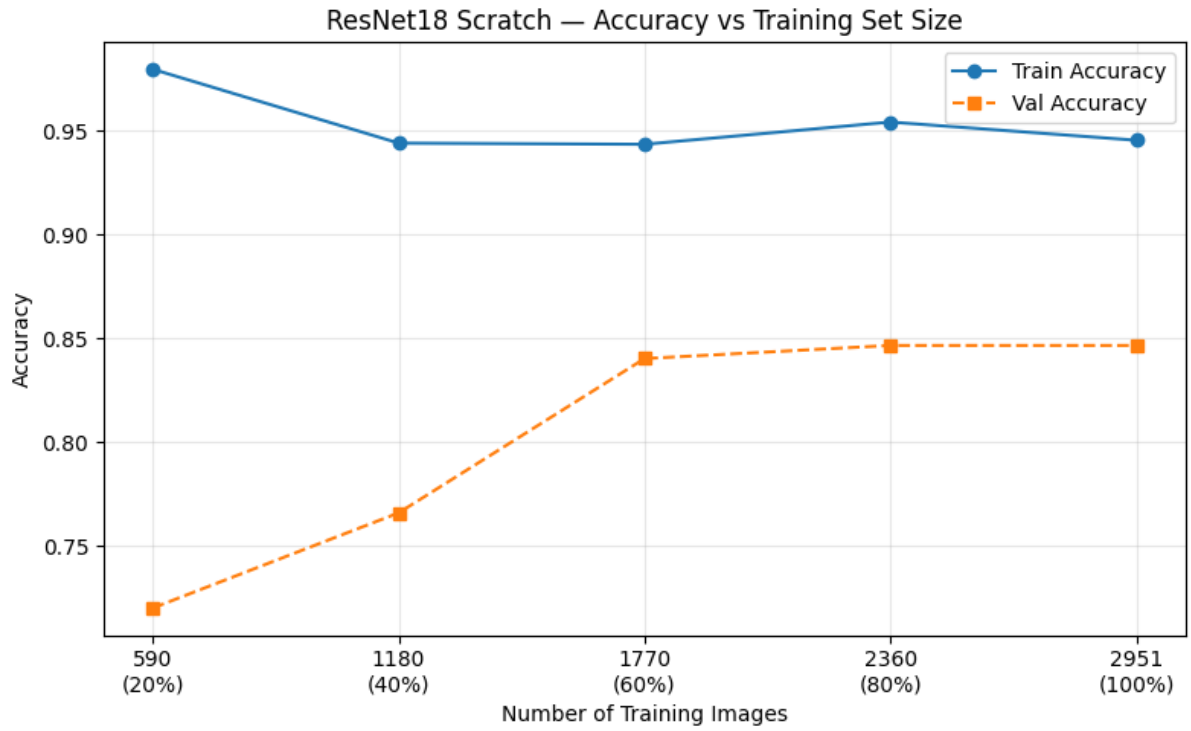
```
=====================================================================
Architecture                    Accuracy  Precision   Recall       F1
=====================================================================
CustomCNN                         0.8328     0.8349   0.8245   0.8223
DeeperCNN                         0.8407     0.8551   0.8444   0.8404
AlexNet Pretrained                0.7744     0.8271   0.7570   0.7486
ResNet18 Scratch                  0.8502     0.8589   0.8439   0.8444
=====================================================================
```

4.2 Over/under-fitting assessment

Model fit was assessed by the difference of the training and validation accuracy at the end of training.

1. Custom CNN (Combo 2) Train Acc: 0.8834 Val Acc: 0.8339 Gap : 0.0495, slight overfitting The model is well fitted with the training data and generalizes with a small loss.
2. Deep CNN (Combo 3) Train Acc: .8983 Val Acc: .8655 Gap: .0328 - very even. SGD with dropout=0.3 gives stable convergence for scratch trained models which is the best gap
3. Pretrained AlexNet (Combination 1) Train Acc = 0.7699 Val Acc = 0.8196 Gap = -0.0497 Slight underfitting. The training accuracy was worse than the validation accuracy as only the last layer was trained and the frozen pre-trained features were not adapted to the domain of retinal image.
4. ResNet18 Scratch (Combo 2)Train Acc = 0.9732Val Acc = 0.8655 Gap = 0.1077 - clearly overfitted The best test accuracy (85.02%) indicates that the model memorized the training patterns despite the large gap. Data augmentation or stronger regularisation can help to close this gap.

Findings are supported by an analysis of the learning curve. 20% training data ResNet18 got train accuracy 97.97% but validation accuracy 71.99% only which shows high variance. Tohe validation accuracy was gradually improved with the increase of training set size, and reached 84.02% at 60% of data.



Testing Methodology

Step 1 – Model Selection: Three hyperparameter combinations were trained for each of the four architectures and the model with the best validation accuracy was selected.

Step 2 Load Test Data The held-out test set of 634 images (15% of the total data set) was loaded. All data was withheld during training and hyper-parameter tuning to prevent biasing the final evaluation.

Step3. Inference: turn off gradient calculation (torch.no_grad()) and set all models to eval mode (model.eval()) The model is fed with test images and the prediction is considered as the class with max softmax output..

Step 4 — Metric Computation: Macro-averaging was used to compute accuracy, precision, recall and F1 score.This method gives equal weight to each of the four disease classes, regardless of the number of samples. In medical classification tasks, where all classes are equally important, macro-averaging is preferred.

Step 5 - Confusion Matrix: For each architecture a confusion matrix was created to show which classes were classified correctly and which classes were confused with each other. The right predictions are on the diagonal.

Step 6 — Comparison: To select the best overall model based on test accuracy and F1 score, a final summary table (Table 4) is provided to compare all four architectures..

The best model for each architecture was tested on the hold-out test set (634 images). The model was evaluated on the basis of four macro averaged metrics i.e. Accuracy, Precision, Recall and F1 Score. We decided to use macro-averaging in order to give equal weight to the four disease classes .

=== CustomCNN -- Per-Class Report ===

	precision	recall	f1-score	support

cataract	0.93	0.75	0.83	170
diabetic_retinopathy	0.97	1.00	0.99	155
glaucoma	0.75	0.63	0.68	141
normal	0.73	0.95	0.83	168
accuracy			0.84	634
macro avg	0.85	0.83	0.83	634
weighted avg	0.85	0.84	0.84	634

=== DeeperCNN -- Per-Class Report ===

	precision	recall	f1-score	support
cataract	0.92	0.78	0.84	170
diabetic_retinopathy	1.00	1.00	1.00	155
glaucoma	0.79	0.65	0.71	141
normal	0.73	0.95	0.83	168
accuracy			0.85	634
macro avg	0.86	0.85	0.85	634
weighted avg	0.86	0.85	0.85	634

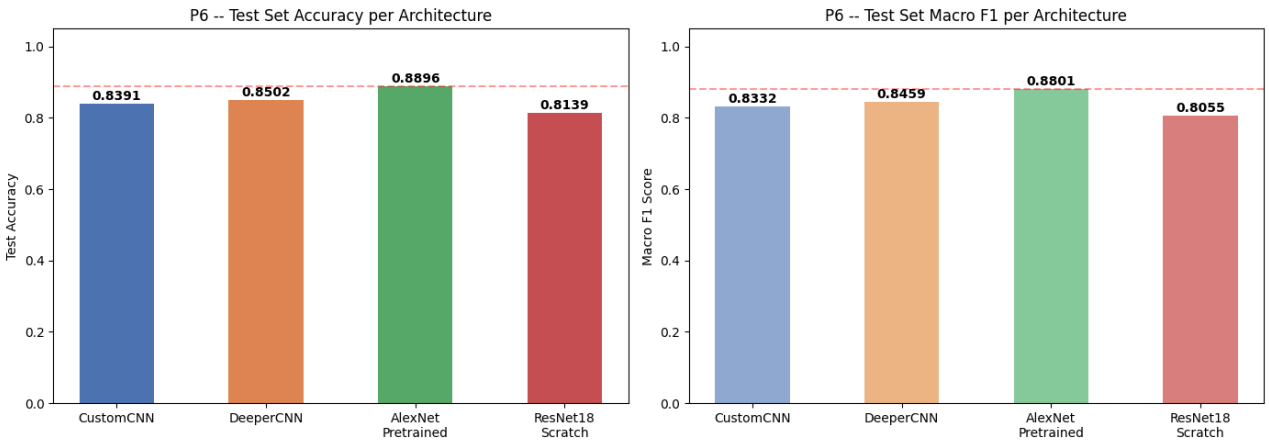
=== AlexNet Pretrained -- Per-Class Report ===

	precision	recall	f1-score	support
cataract	0.88	0.96	0.92	170
diabetic_retinopathy	0.99	1.00	1.00	155
glaucoma	0.93	0.61	0.74	141
normal	0.80	0.95	0.87	168

accuracy			0.89	634
macro avg	0.90	0.88	0.88	634
weighted avg	0.90	0.89	0.88	634

=== ResNet18 Scratch -- Per-Class Report ===

	precision	recall	f1-score	support
cataract	0.89	0.80	0.84	170
diabetic_retinopathy	0.99	0.99	0.99	155
glaucoma	0.80	0.51	0.62	141
normal	0.65	0.92	0.76	168
accuracy			0.81	634
macro avg	0.83	0.81	0.81	634
weighted avg	0.83	0.81	0.81	634



4.3 Results analysis

The trained-from-scratch ResNet18 achieved the best test accuracy of 85.02% and F1 score of 0.8444. Residual connections aid in the efficient flow of gradients during the backpropagation .

Therefore, the model is capable of learning richer feature representations without the need of pre-trained weights.

The second one was Deeper CNN (84.07%) that had the most balanced behavior during training. These results show that the VGG style architecture with moderate depth is a good choice for this size of data.

The simple custom CNN has a decent performance (83.28%) and shows that a well-designed shallow architecture can achieve competitive performance on this task.

AlexNet Pretrained (77.44 %) was the worst performing model. Most likely the feature extraction layers were frozen and could not adapt to the domain of retinal images which is quite different from natural images used for ImageNet pretraining.

The fastest architecture we found out is ResNet18 it had a minimum number of trainable parameters (11,178,564) but lower accuracy. AlexNet has the lowest accuracy and the biggest number of parameters (57,020,228).

4.4 Effectiveness assessment

For each architecture, the best model was picked based on validation accuracy and then tested on the held-out test set (634 images). The best overall model was ResNet18 (Combo 2: Adam, lr=0.0001) with test accuracy of 85.02%, precision of 0.8589, recall of 0.8439 and F1 of 0.8444 .

Precision, recall and F1 were averaged using macro-averaging, which gives equal weight to each class, irrespective of the size of the class . This is helpful for medical classification problems where the disease classes are equally important.

Medical classification accuracy alone is not sufficient as it does not provide information about per-class performance. Recall is important for disease detection as it prevents false negatives (missed diagnoses) and precision prevents unnecessary referrals.The high F1 scores obtained for all architectures support the claim that the models find a good trade-off between those objectives.

4.5 Interface development

The input to this program is the image file path and preprocessing is done using the default preprocessing pipeline (resize the image to 224×224 pixels and scale between -1 and 1). Prediction is done using the best performing model, the ResNet18 Scratch model.

The prediction is plotted on top of the image with class name in green using matplotlib.The best_eye_disease_model is saved on disk.torch.save(model.state_dict(), 'pth') # for future inference without training again.

The interface was tested on sample images and it correctly detected the cases of glaucoma by detecting the typical enlarged optic disc and changed vessel pattern in fundus photography.

4.6 Critical evaluation of models

Lastly, the ResNet18 architecture trained from scratch showed that residual connections are very effective even when pre-trained weights are not available for medical image classification. The analysis of the learning curve shows that the model is very sensitive to the size of the training data.

Transfer learning with AlexNet was worse, as only the last classification layer was fine-tuned. Future work is likely to have much better results either by fine tuning all layers or by using a model pre-trained on medical images (e.g. ChexNet or EyePACS).
Data augmentation (random flips, rotations, colour jitter) would benefit all models, reducing overfitting and improving the generalisation. Moreover, training with a learning rate scheduler for more epochs can improve the performance. Currently, the dataset contains 4217 images and overfitting is a big challenge.

5 References

·, G. V. (2022). *eye_diseases_classification*. Retrieved from https://www.kaggle.com/datasets/gunavenkatdoddi/eye-diseases-classification

Ivana Lucia Kharisma, S. (2025). Implementation of the VGG19 Model with Transfer Learning for Retinal Disease Diagnosis: A Study on Normal Eyes, Diabetic Retinopathy, Cataract, and Glaucoma Datasets. *mbdi*.

Team, G. L. (2020). *AlexNet: The First CNN to win Image Net*. Retrieved from mygreatlearning: https://www.mygreatlearning.com/blog/alexnet-the-first-cnn-to-win-image-net/